

GATE CS/IT: Computer Networks

Error Control

GATE Aspirant

July 23, 2026

Introduction to Error Control

- **The Problem:** As data travels across physical media (copper wires, fiber optics, or wireless radio waves), it is vulnerable to electromagnetic interference, noise, and attenuation.
- **What is an Error?** An error occurs when the signal is altered during transmission, causing a bit to flip from 0 to 1, or from 1 to 0.
- **Types of Errors:**
 - ① **Single-Bit Error:** Only one bit in a given data unit (like a byte or frame) is changed. This is more common in parallel transmission.
 - ② **Burst Error:** Two or more bits in the data unit have changed. The length of the burst is measured from the first corrupted bit to the last corrupted bit. This is highly common in serial transmission and wireless networks.

Detection vs. Correction

- To achieve error control, the sender adds **redundant bits** to the actual data. The receiver uses these extra bits to check the integrity of the message.
- **1. Error Detection:**
 - The receiver can only determine *if* an error has occurred, but not exactly where.
 - Once detected, the receiver typically discards the frame and relies on ARQ (like Go-Back-N or Selective Repeat) to request a retransmission.
 - Requires very few redundant bits (highly efficient).
- **2. Error Correction (Forward Error Correction - FEC):**
 - The receiver uses complex math to find the exact location of the flipped bit(s) and flips them back to their original state without needing a retransmission.
 - Requires a massive amount of redundant bits, consuming significant bandwidth.

Common Error Control Techniques

Technique	Type	Description
Parity Check	Detection	Adds a single bit to make the total number of 1s either even or odd. Cannot detect errors where an even number of bits flip.
Checksum	Detection	Sums up data blocks using 1's complement arithmetic. Primarily used at the Transport/Network layers (TCP/IP).
Cyclic Redundancy Check (CRC)	Detection	Uses polynomial division. Highly robust against burst errors. The standard for the Data Link Layer (Ethernet/Wi-Fi).
Hamming Code	Correction	Uses multiple parity bits overlapping in specific mathematical patterns to pinpoint the exact location of a single-bit error.

Cyclic Redundancy Check (CRC): The Concept

- **What is it?** CRC is the most powerful, robust, and widely used error-detection technique at the Data Link Layer (used in Ethernet, Wi-Fi, etc.).
- **The Mathematical Basis:** Instead of simple addition (like Checksum), CRC treats blocks of binary data as long polynomials. It relies on **Modulo-2 Binary Division**, which is executed at the hardware level using rapid **XOR (Exclusive OR)** operations.
- **The Generator Polynomial ($G(x)$):**
 - Both the sender and receiver must pre-agree on a common divisor called the Generator.
 - It is often represented algebraically. For example, $x^3 + x + 1$ translates to the binary string 1011 (coefficients of x^3, x^2, x^1, x^0).
 - **Crucial Rule:** If the generator binary string has a length of k bits, the highest degree of its polynomial is $k - 1$.

CRC Generation: The Sender's Task

- **Step 1: Appending Zeros.**

- The sender takes the original data message and appends n **zeros** to the end of it.
- The number n is exactly equal to the degree of the generator polynomial (or $k - 1$, where k is the length of the generator string).

- **Step 2: Modulo-2 Division.**

- The sender divides this expanded data string by the Generator string.
- **XOR Math:** Subtraction is done without borrows. ($1 \oplus 1 = 0$, $0 \oplus 0 = 0$, $1 \oplus 0 = 1$, $0 \oplus 1 = 1$).

- **Step 3: The Frame Check Sequence.**

- The final remainder of this division becomes the actual **CRC bits** (also known as the Frame Check Sequence or FCS).
- The sender replaces the appended zeros with this remainder and transmits the frame: **[Data + CRC]**.

CRC Validation: The Receiver's Task

- **Step 1: Receive and Divide.**

- The receiver gets the entire combined frame from the physical wire.
- It performs the exact same Modulo-2 (XOR) division on the entire received string, using the *exact same* Generator polynomial.

- **Step 2: Analyze the Remainder (Syndrome).**

- **Case A (Remainder == 0):** The received frame is perfectly divisible by the generator. The receiver assumes no errors occurred and **accepts** the payload.
- **Case B (Remainder $\neq$ 0):** The bits were flipped or corrupted in transit. The mathematics fail, and the receiver **rejects** (drops) the frame.

- **Why CRC dominates networking:** With a well-chosen polynomial (like CRC-32 used in Ethernet), it detects 100% of single-bit errors, 100% of odd-numbered errors, and 99.9999% of massive burst errors!

- **Problem Statement:** A sender wants to transmit the data message **1101**. The agreed generator polynomial is $G(x) = x^3 + x + 1$. Calculate the transmitted frame and demonstrate how the receiver validates it.
- **Step 1: Convert Polynomial to Binary**
 - $G(x) = 1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x^1 + 1 \cdot x^0$
 - Generator String = **1011**
- **Step 2: Append Zeros**
 - The generator length (k) is 4 bits.
 - We must append exactly $k - 1 = 3$ zeros to the original data.
 - **Dividend:** 1101 appended with 000 $\implies$ **1101000**

CRC Generation: Sender Side

- **Step 3: Modulo-2 Division (XOR)**
- The sender divides the appended data by the generator. Remember: XOR subtraction ($1 \oplus 1 = 0, 0 \oplus 0 = 0, 1 \oplus 0 = 1$).

Quotient (Ignored)

```
-----  
1011 | 1101000  
1011  
----  
1100  
1011  
----  
1110  
1011  
----  
1010  
1011  
----  
001  <-- Remainder (CRC bits)
```

CRC Validation: Receiver Side

- **Step 5: Receiver Division**

- The receiver gets the entire frame (**1101001**) and divides it by the exact same generator (**1011**).

```
-----  
1011 | 1101001  
1011  
----  
1100  
1011  
----  
1110  
1011  
----  
1011  
1011  
----  
000  <-- Remainder (Syndrome)
```

- **Conclusion:** Because the final remainder is exactly **000**, the receiver mathematically proves that no bits were corrupted during transit and successfully accepts the payload!

Checksum: The Concept

- **What is it?** Checksum is a block-based error detection technique. While CRC is standard for the Data Link layer, Checksum is the primary error-detection mechanism for the **Network and Transport layers** (used heavily in TCP, UDP, and IPv4).
- **The Mathematical Basis:** Instead of complex XOR polynomial division, Checksum relies on simple **1's Complement Arithmetic** (binary addition).
- **The Core Idea:**
 - The data is divided into equal-sized blocks or "words" (typically 16-bit segments in TCP/IP).
 - The sender mathematically sums these blocks and sends the inverted result along with the data.
 - The receiver performs the same sum, including the checksum itself, to verify integrity.

Checksum Generation: The Sender's Task

- **Step 1: Segmentation.** The sender divides the complete message into k equal-sized blocks of n bits each (e.g., 16-bit words).
- **Step 2: 1's Complement Addition.**
 - The sender adds all the data blocks together.
 - **Wrap-around rule:** In 1's complement arithmetic, if adding the blocks produces a carry bit that overflows the n -bit size, that carry bit is pulled around and **added back** to the least significant bit (rightmost bit) of the sum.
- **Step 3: Inversion.** The sender takes the final 1's complement (flips all 0s to 1s, and 1s to 0s) of the sum. This inverted sum is the **Checksum**.
- **Step 4: Transmission.** The sender appends the Checksum to the original data blocks and transmits the entire packet.

Checksum Validation: The Receiver's Task

- **Step 1: Receive and Segment.** The receiver gets the packet and divides it into the exact same n -bit blocks, treating the appended Checksum as just another block of data.
- **Step 2: 1's Complement Addition.**
 - The receiver adds *all* the blocks together, including the checksum block, using the same wrap-around carry rule.
- **Step 3: Inversion and Validation.**
 - The receiver takes the 1's complement (inverts) the final total sum.
 - **Case A (Result == 0):** All bits evaluate to 0. The data is intact, and the receiver **accepts** the packet.
 - **Case B (Result $\neq$ 0):** Even a single 1 in the result indicates corruption. The receiver **rejects** the packet.

The Fundamental Limit: The Pigeonhole Principle

- **The Core Problem:** Both Checksum and CRC map a variable, potentially massive payload (e.g., Megabytes or Gigabytes of data) into a strict, fixed-length sequence of bits (like 16, 32, 64, or 128 bits).
- **The Pigeonhole Principle:** Because the number of possible data payloads is infinitely larger than the number of possible checksum/CRC values (2^{16} , 2^{32} , etc.), **collisions are mathematically inevitable**.
- **Collision Definition:** A collision occurs when two completely different data payloads accidentally produce the exact same Checksum or CRC value.
- As the payload size increases, the statistical probability of random noise generating a valid but incorrect Checksum/CRC increases significantly.

Limitations of Checksum with Large Data

- **1. Weakness to Compensating Errors:**

- Checksum relies on simple addition. If one 16-bit word increases by 1 due to noise, and another word decreases by 1, the total sum remains identical. The corruption goes 100% **undetected**.

- **2. Permutation Blindness (Order Independence):**

- Addition is commutative ($A + B = B + A$). If network routing scrambles the order of the 16-bit blocks within the payload, the Checksum will still calculate to the exact same value. It cannot detect reordering.

- **3. Insufficient Bit Space for Large Files:**

- A standard IPv4/TCP header uses a 16-bit checksum ($2^{16} = 65,536$ unique values). For a 10 Megabyte file, the probability of a multi-bit error randomly summing to the correct 16-bit value becomes dangerously high.

Limitations of CRC with Large Data

• 1. The Burst Error Threshold:

- A k -bit CRC (e.g., 32-bit CRC) guarantees the detection of all burst errors of length $\leq k$. However, if a burst error exceeds 32 bits, the detection rate drops to $1 - (1/2^{32})$. While 99.9999% seems high, on high-speed gigabit links processing billions of frames, undetected errors *will* occur over time.

• 2. Hamming Distance Degradation:

- The protective power of a CRC polynomial is inversely proportional to the payload length.
- For example, a specific 32-bit CRC might guarantee a Hamming Distance of 4 (detects any 3 random bit errors) for payloads up to 11,000 bits. If the payload expands beyond that limit, the polynomial weakens, and the Hamming distance drops, meaning 3 random flipped bits might now mathematically bypass the CRC check.

The Modern Solution: Cryptographic Hashing

- **Beyond Simple Error Detection:** When transmitting massive files (like OS images or large databases), simple 16-bit or 32-bit checks are no longer safe.
- **The Upgrade:** Systems shift from Network Layer Checksums to Application Layer **Cryptographic Hash Functions** (like SHA-256 or SHA-512).
- **Why it works:**
 - A 256-bit hash provides 2^{256} possible combinations. This number is larger than the estimated number of atoms in the known universe.
 - Unlike Checksum/CRC, cryptographic hashes are entirely non-linear. Changing even a single bit in a 5 Gigabyte file creates a radically different, unpredictable 256-bit signature, making accidental collisions virtually impossible.